

RELATIONAL DATABASE MODEL

The relational model, introduced by E. F. Codd in 1970, is based on predicate logic and set theory. **Predicate logic**, used extensively in mathematics, provides a framework in which an assertion (statement of fact) can be verified as either true or false. For example, suppose that a student with a student ID of 12345678 is named Melissa Kenneth. This assertion can easily be demonstrated to be true or false. **Set theory** is a mathematical science that deals with sets, or groups of things, and is used as the basis for data manipulation in the relational model. For example, assume that set A contains three numbers: 16, 24, and 77. This set is represented as A (16,24, 77). Furthermore, set B contains four numbers: 44, 77, 90, and 11, and so is represented as B (44, 77, 90, 11).

Given this information, you can conclude that the intersection of A and B yields a result set with a single number, 77. This result can be expressed as $A \cap B = 77$. In other words, A and B share a common value, 77.

Based on these concepts, the relational model has three well-defined components:

1. A logical data structure represented by relations
2. A set of integrity rules to enforce that the data are and remain consistent over time
3. A set of operations that defines how data are manipulated

A Logical View of Data

The DBMS manages and controls access to the data and the database structure. Such an arrangement—placing the DBMS between the application and the database—eliminates most of the file system's inherent limitations. The result of such flexibility, however, is a far more complex physical structure. In fact, the database structures required by both the hierarchical and network database models often become complicated enough to diminish efficient database design. The relational data model changed all of that by allowing the designer to focus on the logical representation of the data and its relationships, rather than on the physical storage details. To use an automotive analogy, the relational database uses an automatic transmission to relieve you of the need to manipulate clutch pedals and gearshifts. In short, the relational model enables you to view data *logically* rather than *physically*.

Tables and Their Characteristics

The logical view of the relational database is facilitated by the creation of data relationships based on a logical construct known as a relation. Because a relation is a mathematical construct, end users find it much easier to think of a relation as a table. A table is perceived as a two-dimensional structure composed of rows and columns. A table is also called a relation because the relational model's creator, E. F. Codd, used the two terms as synonyms. You can think of a table as a *persistent* representation of a logical relation—that is, a relation whose contents can be permanently saved for future use. As far as the table's user is concerned, a table contains *a group of related entity occurrences*—that is, an entity set. For example, a STUDENT table contains a collection of entity occurrences, each representing a student. For that reason, the terms *entity set* and *table* are often used interchangeably.

Characteristics of a Relational Table

1. A table is perceived as a two-dimensional structure composed of rows and columns.
2. Each table row (**tuple**) represents a single entity occurrence within the entity set.
3. Each table column represents an attribute, and each column has a distinct name.
4. Each intersection of a row and column represents a single data value.
5. All values in a column must conform to the same data format.

- Each column has a specific range of values known as the **attribute domain**.
- The order of the rows and columns is immaterial to the DBMS.
- Each table must have an attribute or combination of attributes that uniquely identifies each row.

Table name: STUDENT											Database name: Ch03_TinyCollege
STU_NUM	STU_LNAME	STU_FNAME	STU_INIT	STU_DOB	STU_HRS	STU_CLASS	STU_GPA	STU_TRANSFER	DEPT_CODE	STU_PHONE	PROF_NUM
321452	Bowser	William	C	12-Feb-1985	42	So	2.84	No	BIOL	2134	205
324257	Smithson	Anne	K	15-Nov-1991	81	Jr	3.27	Yes	CIS	2256	222
324258	Brewer	Juliette		23-Aug-1979	36	So	2.26	Yes	ACCT	2256	228
324269	Oblonski	Walter	H	16-Sep-1986	66	Jr	3.09	No	CIS	2114	222
324273	Smith	John	D	30-Dec-1968	102	Sr	2.11	Yes	ENGL	2231	199
324274	Katinga	Raphael	P	21-Oct-1989	114	Sr	3.15	No	ACCT	2267	228
324291	Robertson	Gerald	T	08-Apr-1983	120	Sr	3.87	No	EDU	2267	311
324299	Smith	John	B	30-Nov-1996	15	Fr	2.92	No	ACCT	2315	230

STU_NUM	= Student number
STU_LNAME	= Student last name
STU_FNAME	= Student first name
STU_INIT	= Student middle initial
STU_DOB	= Student date of birth
STU_HRS	= Credit hours earned
STU_CLASS	= Student classification
STU_GPA	= Grade point average
STU_TRANSFER	= Student transferred from another institution
DEPT_CODE	= Department code
STU_PHONE	= 4-digit campus phone extension
PROF_NUM	= Number of the professor who is the student's advisor

Using the STUDENT table shown in the Figure above you can draw the following conclusions corresponding to the points on the characteristics of relational tables:

- The STUDENT table is perceived to be a two-dimensional structure composed of eight rows (tuples) and 12 columns (attributes).
- Each row in the STUDENT table describes a single entity occurrence within the entity set. (The entity set is represented by the STUDENT table.) For example, row 4 in the Figure above describes a student named Walter H. Oblonski. Given the table contents, the STUDENT entity set includes eight distinct entities (rows), or students.
- Each column represents an attribute, and each column has a distinct name.
- All of the values in a column match the attribute's characteristics. For example, the grade point average (STU_GPA) column contains only STU_GPA entries for each of the table rows. Data must be classified according to their format and function. Although various DBMSs can support different data types, most support at least the following:
 - Numeric. You can use numeric data to perform meaningful arithmetic procedures. For example, in the Figure above, STU_HRS and STU_GPA are numeric attributes.
 - Character. Character data, also known as text data or string data, can contain any character or symbol not intended for mathematical manipulation. In the Figure above, STU_CLASS and STU_PHONE are examples of character attributes.
 - Date. Date attributes contain calendar dates stored in a special format known as the Julian date format. In the Figure above, STU_DOB is a date attribute.
 - Logical. Logical data can only have true or false (yes or no) values. In the Figure above, the STU_TRANSFER attribute uses a logical data format.

5. The column's range of permissible values is known as its **domain**. Because the STU_GPA values are limited to the range 0–4, inclusive, the domain is [0,4].
6. The order of rows and columns is immaterial to the user.
7. Each table must have a primary key. In general terms, the **primary key (PK)** is an attribute or combination of attributes that uniquely identifies any given row. In this case, STU_NUM (the student number) is the primary key. Using the data in the Figure above, observe that a student's last name (STU_LNAME) would not be a good primary key because several students have the last name of Smith. Even the combination of the last name and first name (STU_FNAME) would not be an appropriate primary key because more than one student is named John Smith.

Note: Relational database terminology is very precise. Unfortunately, file system terminology sometimes creeps into the database environment. Thus, rows are sometimes referred to as *records*, and columns are sometimes labeled as *fields*. Occasionally, tables are labeled *files*. Technically speaking, this substitution of terms is not always appropriate; the database table is a logical concept rather than a physical concept, and the terms *file*, *record*, and *field* describe physical concepts. Nevertheless, as long as you recognize that the table is actually a logical concept rather than a physical construct, you may think of table rows as records and of table columns as fields. In fact, many database software vendors still use this familiar file system terminology.

KEYS

In the relational model, keys are important because they are used to ensure that each row in a table is uniquely identifiable. They are also used to establish relationships among tables and to ensure the integrity of the data. A **key** consists of one or more attributes that determine other attributes. For example, an invoice number identifies all of the invoice attributes, such as the invoice date and the customer name.

DEPENDENCIES

The role of a key is based on the concept of determination. **Determination** is the state in which knowing the value of one attribute makes it possible to determine the value of another. The idea of determination is not unique to the database environment. You are familiar with the formula $revenue - cost = profit$.

This is a form of determination, because if you are given the *revenue* and the *cost*, you can determine

the *profit*. Given *profit* and *revenue*, you can determine the *cost*.

Given any two values, you can determine the third. Determination in a database environment, however,

is not normally based on a formula but on the relationships among the attributes.

If you consider what the attributes of the STUDENT table in the Figure above actually represent, you will see a relationship among the attributes. If you are given a value for STU_NUM, then you can determine the value for STU_LNAME because one and only one value of STU_LNAME is associated with any given value of STU_NUM. A specific terminology and notation is used to describe relationships based on determination. The relationship is called **functional dependence**, which means that the value of one or more attributes determines the value of one or more other attributes. The standard notation for representing the relationship between STU_NUM and STU_LNAME is as follows:

$STU_NUM \rightarrow STU_LNAME$

In this functional dependency, the attribute whose value determines another is called the **determinant** or the key. The attribute whose value is determined by the other attribute is called the **dependent**. Using this terminology, it would be correct to say that STU_NUM is the determinant and STU_LNAME is the dependent. STU_NUM functionally determines STU_LNAME , and STU_LNAME is functionally dependent on STU_NUM .

As stated earlier, functional dependence can involve a determinant that comprises more than one attribute and multiple dependent attributes. Refer to the STUDENT table for the following example:

$STU_NUM \rightarrow (STU_LNAME, STU_FNAME, STU_GPA)$

And

$(STU_FNAME, STU_LNAME, STU_INIT, STU_PHONE) \rightarrow (STU_DOB, STU_HRS, STU_GPA)$

Determinants made of more than one attribute require special consideration. It is possible to have a functional dependency in which the determinant contains attributes that are not necessary for the relationship. Consider the following two functional dependencies:

$STU_NUM \rightarrow STU_GPA$

$(STU_NUM, STU_LNAME) \rightarrow STU_GPA$

In the second functional dependency, the determinant includes STU_LNAME , but this attribute is not necessary for the relationship. The functional dependency is valid because given a pair of values for STU_NUM and STU_LNAME , only one value would occur for STU_GPA . A more specific term, **full functional dependence**, is used to refer to functional dependencies in which the entire collection of attributes in the determinant is necessary for the relationship. Therefore, the dependency shown in the preceding example is a functional dependency, but not a full functional dependency.

TYPES OF KEYS

A key is an attribute or group of attributes that can determine the values of other attributes. Therefore, keys are determinants in functional dependencies. Several different types of keys are used in the relational model, and you need to be familiar with them.

A **composite key** is a key that is composed of more than one attribute. An attribute that is a part of a key is called a **key attribute**. For example,

$STU_NUM \rightarrow STU_GPA$

$(STU_LNAME, STU_FNAME, STU_INIT, STU_PHONE) \rightarrow STU_HRS$

In the first functional dependency, STU_NUM is an example of a key composed of only one key attribute. In the second functional dependency, $(STU_LNAME, STU_FNAME, STU_INIT, STU_PHONE)$ is a composite key composed of four key attributes.

A **superkey** is a key that can uniquely identify any row in the table. In other words, a superkey functionally determines every attribute in the row. In the STUDENT table,

STU_NUM is a superkey, as are the composite keys (STU_NUM, STU_LNAME), (STU_NUM, STU_LNAME, STU_INIT), and (STU_LNAME, STU_FNAME, STU_INIT, STU_PHONE).

In fact, because STU_NUM alone is a superkey, any composite key that has STU_NUM as a key attribute will also be a superkey.

For example, Gigantic State University determines its student classification based on hours completed, as shown in Table below

HOURS COMPLETED	CLASSIFICATION
Less than 30	Fr
30–59	So
60–89	Jr
90 or more	Sr

Therefore, you can write $STU_HRS \rightarrow STU_CLASS$.

However, the specific number of hours is not dependent on the classification. It is quite possible to find a junior with 62 completed hours or one with 84 completed hours. In other words, the classification (STU_CLASS) does not determine one and only one value for completed hours (STU_HRS).

One specific type of superkey is called a candidate key. A **candidate key** is a minimal superkey—that is, a superkey without any unnecessary attributes. A candidate key is based on a full functional dependency. For example, STU_NUM would be a candidate key, as would (STU_LNAME, STU_FNAME, STU_INIT, STU_PHONE).

On the other hand, (STU_NUM, STU_LNAME) is a superkey, but it is not a candidate key because STU_LNAME could be removed and the key would still be a superkey. A table can have many different candidate keys. If the STUDENT table also included the students' Social Security numbers as STU_SSN, then it would appear to be a candidate key. Candidate keys are called *candidates* because they are the eligible options from which the designer will choose when selecting the primary key. The primary key is the candidate key chosen to be the primary means by which the rows of the table are uniquely identified.

Entity integrity is the condition in which each row (entity instance) in the table has its own unique identity. To ensure entity integrity, the primary key has two requirements:

- (1) all of the values in the primary key must be unique, and
- (2) no key attribute in the primary key can contain a null.

A null is no value at all. It does not mean a zero or a space. A null is created when you press the Enter key or the Tab key to move to the next entry without making an entry of any kind. Pressing the Spacebar creates a blank (or a space).

Null values are problematic in the relational model. A **null** is the absence of any data value, and it is never allowed in any part of the primary key. From a theoretical perspective, it can be argued that a table that contains a null is not properly a relational table at all. From a practical perspective, however, some nulls cannot be reasonably avoided.

For example, not all students have a middle initial. As a general rule, nulls should be avoided as much as reasonably possible. In fact, an abundance of nulls is often a sign of a poor

design. Also, nulls should be avoided in the database because their meaning is not always identifiable. For example, a null could represent any of the following:

- An unknown attribute value
- A known, but missing, attribute value
- A “not applicable” condition

Depending on the sophistication of the application development software, nulls can create problems when functions such as COUNT, AVERAGE, and SUM are used. In addition, nulls can create logical problems when relational tables are linked.

In addition to its role in providing a unique identity to each row in the table, the primary key may play an additional role in the controlled redundancy that allows the relational model to work. A hallmark of the relational model is that relationships between tables are implemented through common attributes as a form of controlled redundancy.

For example, the Figure below shows PRODUCT and VENDOR tables that are linked through a common attribute, VEND_CODE. VEND_CODE is referred to as a foreign key in the PRODUCT table. A **foreign key (FK)** is the primary key of one table that has been placed into another table to create a common attribute. In the Figure below, the primary key of VENDOR, VEND_CODE, was placed in the PRODUCT table; therefore, VEND_CODE is a foreign key in PRODUCT. One advantage of using a proper naming convention for table attributes is that you can identify foreign keys more easily.

Table name: PRODUCT		Database name:		
Primary key: PROD_CODE				
Foreign key: VEND_CODE				
PROD_CODE	PROD_DESCRIPT	PROD_PRICE	PROD_ON_HAND	VEND_CODE
001278-AB	Claw hammer	12.95	23	232
123-21UUY	Houselite chain saw, 16-in. bar	189.99	4	235
QER-34256	Sledge hammer, 16-lb. head	18.63	6	231
SRE-657UG	Rat-tail file	2.99	15	232
ZZX/3245Q	Steel tape, 12-ft. length	6.79	8	235

link

Table name: VENDOR				
Primary key: VEND_CODE				
Foreign key: none				
VEND_CODE	VEND_CONTACT	VEND_AREACODE	VEND_PHONE	
230	Shelly K. Smithson	608	555-1234	
231	James Johnson	615	123-4536	
232	Annelise Crystall	608	224-2134	
233	Candice Wallace	904	342-6567	
234	Arthur Jones	615	123-3324	
235	Henry Ortozo	615	899-3425	

Just as the primary key has a role in ensuring the integrity of the database, so does the foreign key. Foreign keys are used to ensure **referential integrity**, the condition in which every reference to an entity instance by another entity instance is valid. In other words, every foreign key entry must either be null or a valid value in the primary key of the related table. Note that the PRODUCT table has referential integrity because every entry in VEND_CODE in the PRODUCT table is either null or a valid value in VEND_CODE in the VENDOR table. Every vendor referred to by a row in the PRODUCT table is a valid vendor.

Finally, a **secondary key** is defined as a key that is used strictly for data retrieval purposes. Suppose that customer data are stored in a CUSTOMER table in which the customer number is the primary key. Do you think that most customers will remember their numbers? Data retrieval for a customer is easier when the customer's last name and phone number are used. In that case, the primary key is the customer number; the secondary key is the combination of the customer's last name and phone number. Keep in mind that a secondary key does not necessarily yield a unique outcome. For example, a customer's last name and home telephone number could easily yield several matches in which one family lives together and shares a phone line. A less efficient secondary key would be the combination of the last name and zip code; this could yield dozens of matches, which could then be combed for a specific match.

KEY TYPE	DEFINITION
Superkey	An attribute or combination of attributes that uniquely identifies each row in a table
Candidate key	A minimal (irreducible) superkey; a superkey that does not contain a subset of attributes that is itself a superkey
Primary key	A candidate key selected to uniquely identify all other attribute values in any given row; cannot contain null entries
Foreign key	An attribute or combination of attributes in one table whose values must either match the primary key in another table or be null
Secondary key	An attribute or combination of attributes used strictly for data retrieval purposes

INTEGRITY RULES

Relational database integrity rules are very important to good database design.

ENTITY INTEGRITY	DESCRIPTION
Requirement	All primary key entries are unique, and no part of a primary key may be null.
Purpose	Each row will have a unique identity, and foreign key values can properly reference primary key values.
Example	No invoice can have a duplicate number, nor can it be null; in short, all invoices are uniquely identified by their invoice number.
REFERENTIAL INTEGRITY	DESCRIPTION
Requirement	A foreign key may have either a null entry, as long as it is not a part of its table's primary key, or an entry that matches the primary key value in a table to which it is related; (every non-null foreign key value <i>must</i> reference an <i>existing</i> primary key value).
Purpose	It is possible for an attribute <i>not</i> to have a corresponding value, but it will be impossible to have an invalid entry; the enforcement of the referential integrity rule makes it impossible to delete a row in one table whose primary key has mandatory matching foreign key values in another table.
Example	A customer might not yet have an assigned sales representative (number), but it will be impossible to have an invalid sales representative (number).

To avoid nulls, some designers use special codes, known as **flags**, to indicate the absence of some value. Using Figure below as an example, the code -99 could be used as the AGENT_CODE entry in the fourth row of the CUSTOMER table to indicate that customer Paul Olowski does not yet have an agent assigned to him. If such a flag is used, the AGENT table must contain a dummy row with an AGENT_CODE value of -99. Thus, the AGENT table's first record might contain the values shown in Table below.

AGENT_CODE	AGENT_AREACODE	AGENT_PHONE	AGENT_LNAME	AGENT_YTD_SLS
-99	000	000-0000	None	\$0.00

Table name: CUSTOMER

Database name: Ch03_InsureCo

Primary key: CUS_CODE

Foreign key: AGENT_CODE

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_RENEW_DATE	AGENT_CODE
10010	Ramas	Alfred	A	05-Apr-2014	502
10011	Dunne	Leona	K	16-Jun-2014	501
10012	Smith	Kathy	vV	29-Jan-2015	502
10013	Olowski	Paul	F	14-Oct-2014	502
10014	Orlando	Myron		28-Dec-2014	501
10015	O'Brian	Amy	B	22-Sep-2014	503
10016	Brown	James	G	25-Mar-2015	502
10017	Williams	George		17-Jul-2014	503
10018	Farriss	Anne	G	03-Dec-2014	501
10019	Smith	Olette	K	14-Mar-2015	503

Table name: AGENT (only five selected fields are shown)

Primary key: AGENT_CODE

Foreign key: none

AGENT_CODE	AGENT_AREACODE	AGENT_PHONE	AGENT_LNAME	AGENT_YTD_SLS
501	713	228-1249	Alby	132735.75
502	615	882-1244	Hahn	138967.35
503	615	123-5589	Okon	127093.45

Other integrity rules that can be enforced in the relational model are the NOT NULL and UNIQUE constraints. The NOT NULL constraint can be placed on a column to ensure that every row in the table has a value for that column. The UNIQUE constraint is a restriction placed on a column to ensure that no duplicate values exist for that column.